

# Dynamics and Stability of the Spring-Loaded Inverted Pendulum (SLIP) Model

## 1 Introduction and Relevance

The spring-loaded inverted pendulum (SLIP) is the reduced-order model behind most legged-robot dynamics [3, 5]: a point mass bouncing on a massless spring leg. Despite its simplicity, it captures the center-of-mass dynamics of running in animals and machines remarkably well [1, 2]. That makes it a good subject for this course: simple enough to analyze fully with the tools we covered, but rich enough to produce interesting hybrid dynamics, and it builds directly on the elastic pendulum studied in class, since the stance phase of SLIP is an inverted spring pendulum.

SLIP is mechanically interesting as a hybrid dynamical system: during stance the body behaves like a spring-loaded pendulum, while in flight it is a free projectile, with the two phases connected by touchdown and liftoff switching conditions. For a hopping gait, the natural stability question is not about a fixed equilibrium but about whether the periodic motion itself is stable. The standard tool for that is the apex return map, a Poincaré map in which a steady hopping gait appears as a fixed point whose stability can be assessed numerically [1, 4].

Beyond being a convenient system, SLIP-type models are used directly in the design and control of running robots and in the biomechanical study of animal and human locomotion – my practical motivation for this project: a model simple enough to analyze by hand but predictive enough to inform how a real legged system should be built or controlled.

### Relation to Prior Work

This project works within an established line of SLIP research; each stage of the model draws on a different part of that literature. Blickhan [1] introduced the spring-mass template and showed it reproduces the ground-reaction-force and center-of-mass patterns of human and animal running; the stance-phase Lagrangian derived here reproduces the same model, but from Lagrange’s equations in polar coordinates rather than Blickhan’s Newtonian force balance. McMahon and Cheng [2] used the same template to relate leg stiffness to running speed empirically; the parameter sweep over stiffness  $k$  and touchdown angle  $\theta_{TD}$  planned here asks the same question numerically, from the model directly, rather than from experimental data. Full and Koditschek’s templates-and-anchors framework [3] justifies treating SLIP as a legitimate object of study on its own, not just a shortcut toward a fully articulated multi-link leg model: a template is expected to capture the essential dynamics of the anchored system while discarding detail that does not matter for stability. Schwind and Koditschek [4] derived an approximate, closed-form stance map using a perturbative solution; this project instead integrates the stance-phase equations numerically with an event-based solver, trading their analytical insight for an exact (if less interpretable) stance map, consistent with this course’s computational emphasis. Saranli, Buehler, and Koditschek [5] built RHex, a hexapod whose legs behave like a SLIP template during stance – the practical grounding for why this abstraction is worth studying: not just a toy system, but the model actually used to understand a working robot. Finally, Geyer, Seyfarth, and Blickhan [6] extended a SLIP-type compliant-leg model to walking, not just running, by allowing double support – a natural direction for future work beyond the single-support hopping model considered here.

## 2 Modeling of Dynamics

The equations of motion for both phases are derived using **Lagrange’s equations of motion**, with generalized coordinates chosen separately per phase: Cartesian coordinates  $(x, y)$  for flight, and polar

coordinates  $(r, \theta)$  about the planted foot for stance. The two phases are connected by touchdown and liftoff switching (transversality) conditions, so the overall model is a hybrid dynamical system integrated with event-based numerical methods. Figure 1 shows the geometry of both phases.

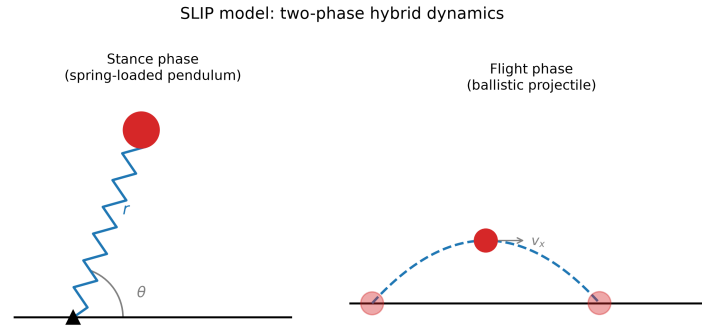


Figure 1: SLIP model geometry: stance (spring pendulum,  $r, \theta$ ) and flight (projectile).

## 2.1 Flight-Phase Equations of Motion

With basis  $\hat{n}_1, \hat{n}_2$ ,  $\vec{r}_m = x\hat{n}_1 + y\hat{n}_2$ :

$$T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2), \quad V = mgy, \quad L = T - V = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) - mgy$$

Lagrange's equations give

$$\ddot{x} = 0, \quad \ddot{y} = -g$$

Starting at the apex ( $\dot{y} = 0$ ), with  $x(0) = x_0$ ,  $\dot{x}(0) = v_{x0}$ :

$$x(t) = v_{x0}t + x_0, \quad y(t) = y_0 - \frac{g}{2}t^2$$

## 2.2 Touchdown: Detection and Transition Map

$\theta_{TD}$  is held fixed in flight, so the foot trails the body at

$$\vec{r}_{foot}(t) = (x(t) - r_0 \cos \theta_{TD})\hat{n}_1 + (y(t) - r_0 \sin \theta_{TD})\hat{n}_2$$

Touchdown is  $y_{foot} = 0$ , i.e.  $y_{TD} = r_0 \sin \theta_{TD}$ . Solving  $y(t) = y_0 - \frac{g}{2}t^2$ :

$$t_{TD} = \sqrt{\frac{2(y_0 - r_0 \sin \theta_{TD})}{g}}, \quad \dot{y}_{TD} = -\sqrt{2g(y_0 - r_0 \sin \theta_{TD})}, \quad x_{TD} = v_{x0}t_{TD}, \quad \dot{x}_{TD} = v_{x0}$$

Stance uses polar coordinates about the planted foot, so this Cartesian state must be converted. With foot at  $\vec{r}_F = (x_{TD} + r_0 \cos \theta_{TD})\hat{n}_1$  and leg vector  $\vec{\rho} = \vec{r}_m - \vec{r}_F = -r_0 \cos \theta_{TD} \hat{n}_1 + y_{TD} \hat{n}_2$ :

$$r = |\vec{\rho}| = \sqrt{r_0^2 \cos^2 \theta_{TD} + y_{TD}^2} = r_0, \quad \theta = \text{atan2}(y_{TD}, -r_0 \cos \theta_{TD})$$

Projecting  $\vec{v} = \dot{x}_{TD}\hat{n}_1 + \dot{y}_{TD}\hat{n}_2$  onto  $\hat{e}_r = \cos \theta \hat{n}_1 + \sin \theta \hat{n}_2$ ,  $\hat{e}_\theta = -\sin \theta \hat{n}_1 + \cos \theta \hat{n}_2$ :

$$\dot{r} = \vec{v} \cdot \hat{e}_r = \dot{x}_{TD} \cos \theta + \dot{y}_{TD} \sin \theta, \quad \dot{\theta} = \frac{\vec{v} \cdot \hat{e}_\theta}{r_0} = \frac{-\dot{x}_{TD} \sin \theta + \dot{y}_{TD} \cos \theta}{r_0}$$

implemented as `touchdownState()`.

## 2.3 Stance-Phase Equations of Motion

In stance the leg pivots about the fixed foot:  $\vec{r}_m = r\hat{e}_r$ ,  $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$ :

$$T = \frac{1}{2}m\dot{r}^2 + \frac{1}{2}mr^2\dot{\theta}^2, \quad V = mgr \sin \theta + \frac{1}{2}k(r - r_0)^2, \quad L = T - V$$

Lagrange's equations:

$$m\ddot{r} - mr\dot{\theta}^2 + mg \sin \theta + k(r - r_0) = 0, \quad mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} + mgr \cos \theta = 0$$

solved for the highest derivatives:

$$\ddot{r} = r\dot{\theta}^2 - g \sin \theta - \frac{k}{m}(r - r_0), \quad \ddot{\theta} = -\frac{2}{r}\dot{r}\dot{\theta} - \frac{g}{r} \cos \theta$$

implemented as `standingState()`.

## 2.4 Conserved Energy in Stance

$\vec{r}_m = r\hat{e}_r$  is scleronomic and  $T$  is homogeneous quadratic in  $(\dot{r}, \dot{\theta})$ , so  $H = T + V$ ; with no explicit  $t$ -dependence, energy is conserved through stance:

$$E = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr \sin \theta + \frac{1}{2}k(r - r_0)^2 = \text{const.}$$

Neither coordinate is cyclic, so both equations must be integrated together; this conserved quantity is used below as a check on the numerical integrator.

## 2.5 Liftoff Map and the Apex Return Map

Liftoff occurs at  $r = r_0$ ,  $\dot{r} > 0$ . Chaining flight  $\rightarrow$  touchdown  $\rightarrow$  stance  $\rightarrow$  liftoff  $\rightarrow$  flight once defines the apex-to-apex map

$$P : (y_n, v_{x,n}) \mapsto (y_{n+1}, v_{x,n+1}),$$

whose fixed points are steady periodic hopping gaits and whose stability is governed by the linearization of  $P$  there. Section 2.7 reduces this to a single scalar, so that linearization is one derivative rather than a Jacobian.

Liftoff position converts to Cartesian from  $\vec{r}_m = r\hat{e}_r$  at  $r = r_0$ ,  $\theta = \theta_{LO}$ :

$$x_{LO} = r_0 \cos \theta_{LO}, \quad y_{LO} = r_0 \sin \theta_{LO}$$

Velocity converts by substituting  $\hat{e}_r = \cos \theta \hat{n}_1 + \sin \theta \hat{n}_2$ ,  $\hat{e}_\theta = -\sin \theta \hat{n}_1 + \cos \theta \hat{n}_2$  into  $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$  and evaluating at liftoff:

$$\dot{x}_{LO} = \dot{r}_{LO} \cos \theta_{LO} - r_0 \dot{\theta}_{LO} \sin \theta_{LO}, \quad \dot{y}_{LO} = \dot{r}_{LO} \sin \theta_{LO} + r_0 \dot{\theta}_{LO} \cos \theta_{LO}$$

Once airborne, apex is defined in the ground frame by  $\dot{y} = 0$ . With  $\ddot{y} = -g$  and the clock started at liftoff,  $\dot{y}(t) = \dot{y}_{LO} - gt$ , so

$$t_{\text{apex}} = \frac{\dot{y}_{LO}}{g}$$

Propagating the flight solution  $x(t) = \dot{x}_{LO}t + x_{LO}$ ,  $y(t) = y_{LO} + \dot{y}_{LO}t - \frac{g}{2}t^2$  to  $t = t_{apex}$ :

$$x_{n+1} = x_{LO} + \dot{x}_{LO} t_{apex}, \quad y_{n+1} = y_{LO} + \dot{y}_{LO} t_{apex} - \frac{g}{2} t_{apex}^2, \quad v_{x,n+1} = \dot{x}_{LO}, \quad \dot{y}_{n+1} = 0$$

Both are exact:  $v_{x,n+1} = \dot{x}_{LO}$  since horizontal motion is unforced in flight ( $\ddot{x} = 0$ ), and  $\dot{y}_{n+1} = 0$  by construction of  $t_{apex}$ .  $x_{LO}$  is measured relative to the stance foot just released rather than a fixed ground origin. This is consistent, since  $x$  never enters the dynamics of either phase (Section 2.7).

## 2.6 Conserved Energy at Apex and Across a Full Hop Cycle

By the same argument as Section 2.4, the flight Lagrangian is scleronomic with  $T$  homogeneous quadratic in  $(\dot{x}, \dot{y})$ , so  $H = T + V$  is conserved through flight:

$$E = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) + mgy = \text{const.}$$

with no spring term, since the leg carries no elastic energy while airborne. This differs from the stance energy  $E_{stance} = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr \sin \theta + \frac{1}{2}k(r - r_0)^2$  only by that spring term, which vanishes at  $r = r_0$  – exactly the touchdown and liftoff condition – so the two functionals agree at every switching event: total mechanical energy is continuous across touchdown and liftoff, and conserved over the full cycle apex  $\rightarrow$  touchdown  $\rightarrow$  stance  $\rightarrow$  liftoff  $\rightarrow$  apex (validated numerically in Section 4.2).

## 2.7 Dimension Reduction of the Apex State

At apex,  $\dot{y} = 0$  by definition. Substituting into the apex energy of Section 2.6 and solving for  $\dot{x}$ :

$$E - mgy = \frac{1}{2}m\dot{x}^2 \implies \dot{x}^2 = \frac{2(E - mgy)}{m} = \frac{2E}{m} - 2gy$$

$$\dot{x} = \sqrt{\frac{2E}{m} - 2gy} \quad (\dot{x} > 0 \text{ for forward hopping})$$

So the apex state  $(x, y, \dot{x}, \dot{y})$  is not four independent numbers:  $\dot{y} = 0$  always,  $x$  is ignorable (it enters no potential or force in either phase), and  $\dot{x}$  is fixed by  $y$  once  $E$  is fixed, collapsing the apex state to the scalar  $y$  and reducing the hop-to-hop dynamics to the one-dimensional map

$$y_{n+1} = P(y_n)$$

where  $P$  is the composition: reconstruct  $\dot{x}_n$  from  $y_n$  (above)  $\rightarrow$  flight to touchdown (Section 2.2)  $\rightarrow$  stance to liftoff (Section 2.3)  $\rightarrow$  flight to the next apex (Section 2.5). Section 4.2 confirms this reduction numerically:  $\dot{x}$  evaluated at the simulated output height  $y_{n+1}$  matches the simulator's own  $\dot{x}_{n+1}$  to machine precision.

## 2.8 The Apex Return Map: Fixed Point and Stability Criterion

Using the reduction of Section 2.7, define the apex return map  $P$  at fixed leg stiffness  $k$ , fixed touchdown angle  $\theta_{TD}$ , and fixed total energy  $E$ :

$$P : y_n \mapsto y_{n+1}$$

$E$  characterizes how energetic a gait is, and  $k$ ,  $\theta_{TD}$  are the model's only free parameters; with no controller, a periodic gait at a given  $E$  is whatever  $P$  produces for a chosen  $(k, \theta_{TD})$ .

A periodic gait is a fixed point  $y^* = P(y^*)$ , equivalently a root of

$$f(y) = P(y) - y$$

Local stability is governed by the derivative at the fixed point, estimated by a central finite difference:

$$P'(y^*) \approx \frac{P(y^* + dy) - P(y^* - dy)}{2 dy}$$

p

$$|P'(y^*)| < 1 : \text{stable}, \quad |P'(y^*)| > 1 : \text{unstable}$$

$P'(y^*) < 0$  indicates oscillatory divergence/convergence (perturbations alternate sides of  $y^*$ );  $P'(y^*) > 0$  indicates monotonic divergence/convergence. Section 4.3 reports the root-finding methodology and the result for the validated parameter set of Section 4.1.

### 3 Discussion of Assumptions

Several modeling choices were made to keep the SLIP model tractable with the tools of this course, each with a corresponding limitation:

- **Planar motion.** The model is restricted to the sagittal plane; real running involves lateral and rotational trunk/leg motion this model cannot capture.
- **Point mass, massless leg.** All body mass is lumped at a single point, and the leg carries no mass or rotational inertia. This isolates the center-of-mass dynamics SLIP is meant to describe, but discards leg-swing inertia and trunk pitch – detail the templates-and-anchors view [3] explicitly allows a template to omit.
- **Linear (Hookean) leg spring.** The stance leg is modeled as a single ideal linear spring,  $F = -k(r - r_0)$ , with constant stiffness  $k$ . Real leg stiffness is nonlinear and can be actively modulated by the neuromuscular system or a controller, so treating  $k$  as a fixed swept parameter is a simplification.
- **Fixed touchdown angle held through flight.**  $\theta_{TD}$  is held constant through flight – no swing-leg retraction or angle adjustment before the foot lands. This is the standard SLIP assumption, but it means the model’s only “control input” between hops is the touchdown angle chosen before each flight, not a continuous in-flight correction.
- **Instantaneous, rigid, non-slipping foot placement.** Touchdown and liftoff are instantaneous events with no impact losses, and the foot sticks to the ground (no slipping, no compliance) at a fixed height  $y = 0$  through stance. Real contact has finite compliance and a friction cone that could, in principle, be violated at high touchdown angles or speeds.
- **No stance-phase dissipation, no aerodynamic drag.** The stance-phase Lagrangian has no damping term, so mechanical energy is exactly conserved within a stance phase (verified numerically in Section 4); flight is likewise drag-free projectile motion. This is deliberate: it isolates the elastic dynamics of interest and gives a clean, near-machine-precision integrator check, at the cost of ignoring the energy any real leg or running surface dissipates each stride.
- **Equal rest length at touchdown and liftoff.** The leg returns to the same natural length  $r_0$  between hops, with no active extension/retraction actuation changing rest length from one stance phase to the next.

## 4 Computational Results

### 4.1 Single-Stance Validation

The model is implemented in Python. `main.py` maps an apex state to the stance-phase initial condition via `touchdownState()`, then integrates `standingState()` with `solve_ivp` ( $\text{rtol} = 10^{-10}$ ,  $\text{atol} = 10^{-12}$ ) to the liftoff event  $r = r_0$ ,  $\dot{r} > 0$ , checking the conserved energy from Section 2.4 at every step. Using the parameters below, this validates the stance-phase model end to end:

| $m$   | $r_0$ | $k$       | $g$                   | $\theta_{TD}$ | $y_0$ | $v_{x0}$ |
|-------|-------|-----------|-----------------------|---------------|-------|----------|
| 80 kg | 1 m   | 20000 N/m | 9.81 m/s <sup>2</sup> | 68°           | 1.2 m | 3.0 m/s  |

|           | $r$ [m] | $\theta$ [deg] | $\dot{r}$ [m/s] | $\dot{\theta}$ [rad/s] |
|-----------|---------|----------------|-----------------|------------------------|
| Touchdown | 1.0000  | 112.00         | −3.2689         | −1.9149                |
| Liftoff   | 1.0000  | 83.48          | 3.1937          | −1.6886                |

Stance lasts 0.2094 s; the leg compresses to 0.7740 m (22.6% of  $r_0$ ); relative energy drift is  $6.46 \times 10^{-13}$  (machine precision), confirming the integrator and Section 2.3's equations of motion are consistent. Figure 2 plots leg length, angle, CoM trajectory, and energy drift for this run.

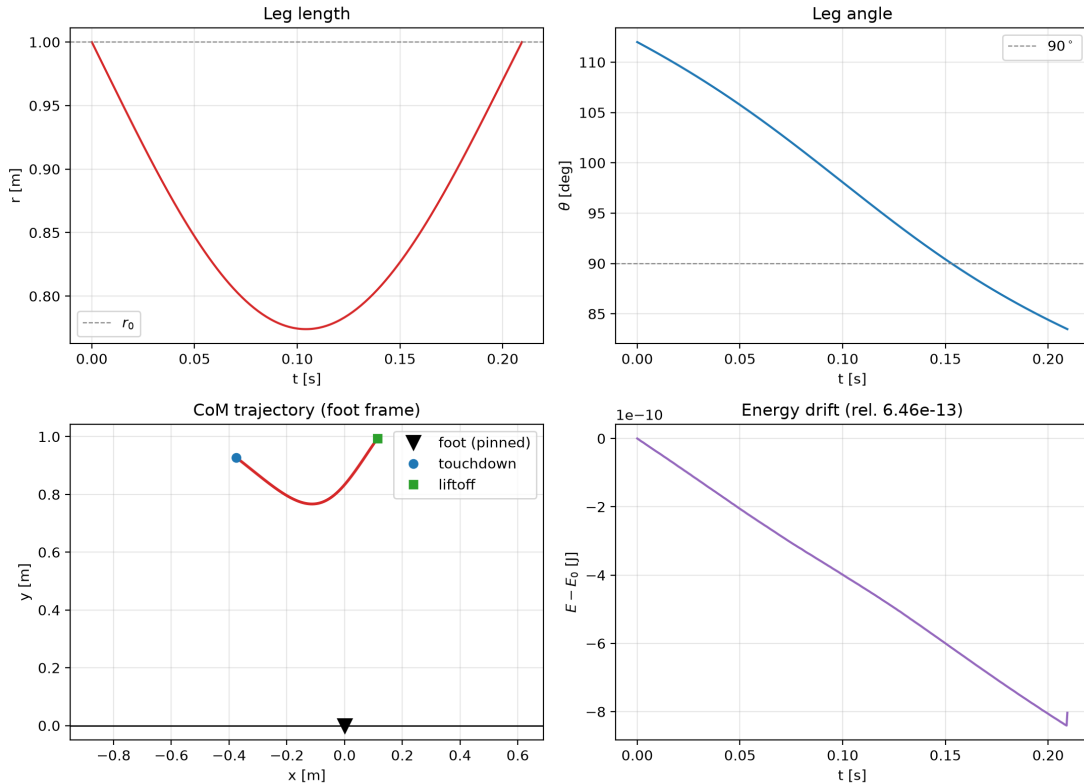


Figure 2: Stance simulation:  $r(t)$ ,  $\theta(t)$  (top), CoM trajectory (bottom left), energy drift (bottom right).

### 4.2 Full-Cycle Energy Conservation and Dimension-Reduction Validation

`proveX_dotIsRedundant()` validates the full hop cycle: starting from the apex state of Section 4.1, it computes the apex energy  $E_1$  (`energyHop()`), chains one hop – flight  $\rightarrow$  touchdown  $\rightarrow$  stance  $\rightarrow$  liftoff

→ flight (`hop()`) – and recomputes the apex energy  $E_2$ . Relative drift over the full cycle,

$$\frac{|E_2 - E_1|}{|E_1|} = 6.17 \times 10^{-13},$$

is consistent with the stance-only drift of Section 4.1 ( $6.46 \times 10^{-13}$ ), confirming total mechanical energy is conserved across the full switching cycle, not just within stance.

The same run checks the dimension reduction of Section 2.7: with  $E_1$  fixed,  $\dot{x} = \sqrt{2E_1/m - 2gy}$  evaluated at the simulated output height  $y_{n+1} = 1.4465$  m predicts  $\dot{x}_{pred} = 2.0403$  m/s, matching the simulator's own  $v_{x,n+1} = 2.0403$  m/s to

$$\frac{|v_{x,n+1} - \dot{x}_{pred}|}{|\dot{x}_{pred}|} = 2.41 \times 10^{-12}.$$

### 4.3 Apex Return Map: Root-Finding Methodology and Result

`findApexMapFixedPoint()` locates the root of  $f(y) = P(y) - y$  (Section 2.8) over the domain of  $y$  physically valid for a given  $(k, \theta_{TD}, E)$ : bounded below by the minimum apex height consistent with touchdown at angle  $\theta_{TD}$ ,

$$y_{min} = r_0 \sin \theta_{TD},$$

and above by the height at which the closed-form  $\dot{x}$  of Section 2.7 would require a negative value under the square root,

$$y_{max} = \frac{E}{mg}.$$

A small inward padding ( $\epsilon = 10^{-3}$  m) is added to  $y_{min}$  to avoid floating-point roundoff pushing the touchdown free-fall relation's square root slightly negative at the exact boundary.

$f(y)$  is sampled coarsely (30 points) across  $[y_{min}, y_{max}]$ ; once a bracket with a sign change is found, Brent's method refines it to a root  $y^*$ , and  $P'(y^*)$  is estimated by the central difference of Section 2.8 with  $dy = 10^{-3}$  m.

For the validated parameter set of Section 4.1 ( $m = 80$  kg,  $r_0 = 1$  m,  $k = 20000$  N/m,  $\theta_{TD} = 68^\circ$ ,  $E$  fixed by  $y_0 = 1.2$  m,  $v_{x0} = 3.0$  m/s):

$$y^* \approx 1.5096 \text{ m}, \quad P'(y^*) \approx -2.76$$

$|P'(y^*)| > 1$ : this fixed point is unstable, and the negative sign indicates oscillatory divergence – a perturbed apex height overshoots on alternating sides of  $y^*$ , growing each hop. Steady hopping at  $\theta_{TD} = 68^\circ$ ,  $k = 20000$  N/m is therefore not stable.

### 4.4 Grid Sweep Methodology and Parameter Sweep

`gridSearchFixedPoints()` wraps the pipeline of Section 4.3 into a function of  $(k, \theta_{TD})$  returning  $(y^*, P'(y^*))$ , or null if  $f(y)$  has no sign change – i.e. no periodic gait exists there at this  $E$ . Each cell uses its own copy of the parameter dictionary, so sweeping  $k$  never mutates state shared across cells. `stableBandWidth()` additionally reports the longest run of contiguous stable cells along each grid direction, used below to tell a genuinely narrow stability margin from a coarse-grid artifact.

An initial coarse  $40 \times 40$  diagnostic grid over  $k \in [5000, 40000]$  N/m,  $\theta_{TD} \in [55^\circ, 80^\circ]$  turned up only a one-cell-wide staircase of stable cells (band width 2 cells along  $\theta_{TD}$ , 3 along  $k$ ) through an otherwise unstable field – too thin to trust at that resolution. `localRefinementSweep()` zoomed a  $20 \times 20$  window ( $\pm 2.5^\circ \times \pm 2500$  N/m, roughly  $4 \times$  finer) onto a representative stable cell near  $k \approx 20256$

N/m,  $\theta_{TD} \approx 71.0^\circ$ ; there the band *thickened* (3 cells along  $\theta_{TD}$ , 5 along  $k$ ), meaning the coarse staircase was a grid-resolution artifact, so the full sweep needed that finer spacing to be trustworthy.

The final grid matches that resolution across the whole domain: a  $133 \times 95$  sweep ( $k \in [5000, 40000]$  N/m in steps of  $\approx 265$  N/m,  $\theta_{TD} \in [55^\circ, 80^\circ]$  in steps of  $\approx 0.27^\circ$ ; 12,635 cells, only 5 with no fixed point), classified stable/unstable/no-fixed-point in Figure 3, with the friction-cone floor from Section 4.3 ( $\theta_{TD} \geq 59^\circ$ ,  $\mu = 0.6$ ) overlaid. The band widens slightly here (3 cells along  $\theta_{TD}$ , 8 along  $k$ ) but is still only a few cells across. A second local refinement at the same fine spacing (Figure 4), centered on the final grid's own representative stable cell ( $k \approx 20114$  N/m,  $\theta_{TD} \approx 70.96^\circ$ ,  $y^* \approx 0.9496$  m,  $P'(y^*) \approx 0.348$ ), gives a band width of 3 cells along  $\theta_{TD}$  and 5 along  $k$  – the same order as the full grid, so the earlier thickening has stopped: the self-stable region is a real, if narrow, sliver of  $(k, \theta_{TD})$  space at this  $E$ , not a grid-spacing artifact.

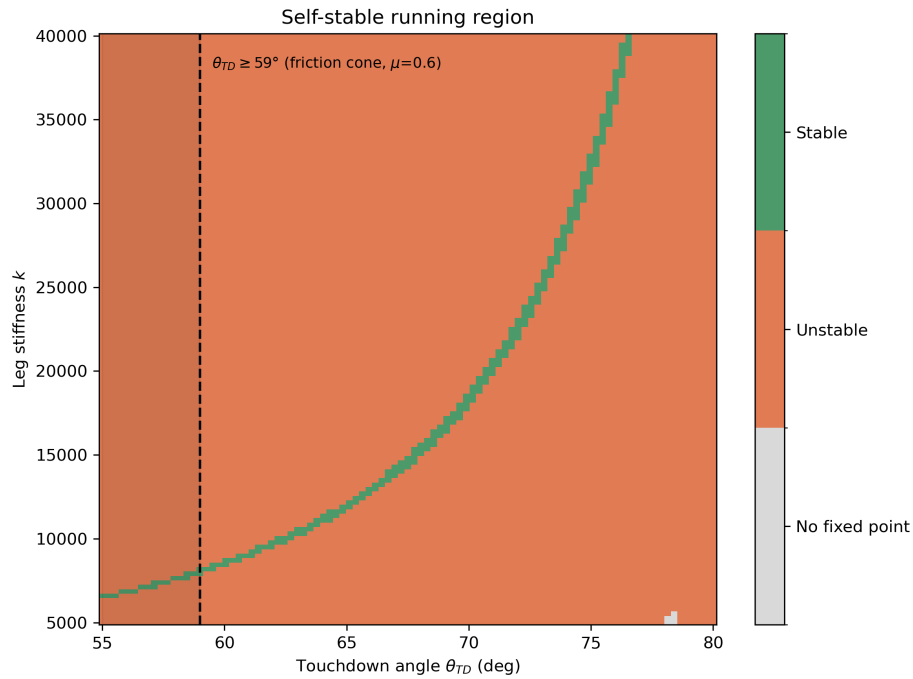


Figure 3: Self-stable running region over the full  $133 \times 95$  ( $k, \theta_{TD}$ ) grid at fixed  $E$  (Section 4.1 parameters). Green: stable fixed point ( $|P'(y^*)| < 1$ ); orange: unstable; gray: no fixed point found. Dashed line/shading: the friction-cone floor  $\theta_{TD} \geq 59^\circ$  ( $\mu = 0.6$ ).



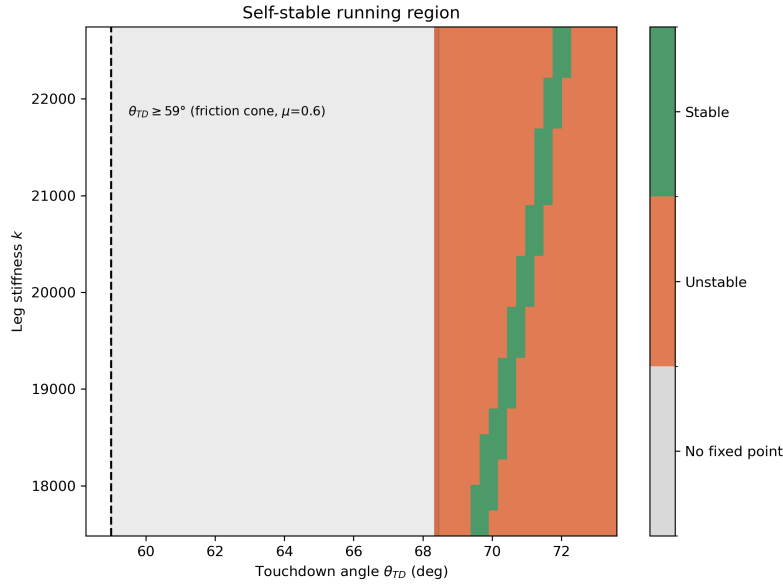


Figure 4: Local refinement ( $20 \times 20$ ,  $\pm 2.5^\circ \times \pm 2500$  N/m) around the stable band near  $k \approx 20114$  N/m,  $\theta_{TD} \approx 70.96^\circ$ , at the same fine spacing used to validate Figure 3's resolution. The band's width here matches the full grid, confirming it is not still thickening.

#### 4.5 Apex-Map Iteration: Cobweb Diagrams for the Stable and Unstable Cases

Section 4.3 and Section 4.4 each report a fixed point and a slope, but neither shows what convergence or divergence looks like hop to hop. `plotApexMapIteration()` does that directly by iterating  $y_{n+1} = P(y_n)$  from a perturbed initial height and plotting both the cobweb diagram (the map  $P(y)$ , the diagonal  $y_{n+1} = y_n$ , and the resulting staircase) and  $y_n$  vs. hop index  $n$ , for two cases at the same fixed energy  $E$ : the stable representative cell of Section 4.4 ( $k \approx 20114$  N/m,  $\theta_{TD} \approx 70.96^\circ$ ,  $y^* \approx 0.9496$  m,  $P'(y^*) \approx 0.35$ ) and the unstable validated point of Section 4.3 ( $k = 20000$  N/m,  $\theta_{TD} = 68^\circ$ ,  $y^* \approx 1.5096$  m,  $P'(y^*) \approx -2.76$ ).

Figure 5 confirms the qualitative prediction of Section 2.8 from the sign and magnitude of  $P'(y^*)$  alone: perturbing the stable case by  $+0.05$  m produces a monotonic staircase that steps down the same side of the diagonal every hop, converging smoothly onto  $y^*$ . This monotonic decay only shows up for a *small* perturbation, which is itself informative: the sweep only certifies *local* stability, and a larger perturbation instead wanders through the map's global nonlinearity (the hump shape of  $P(y)$  visible in the figure) before it happens to land back near  $y^*$ , if it does at all. Perturbing the unstable case by only  $+0.02$  m, by contrast, produces a staircase that immediately crosses to the other side of the diagonal every hop and spirals outward, consistent with the oscillatory divergence already reported in Section 4.3.

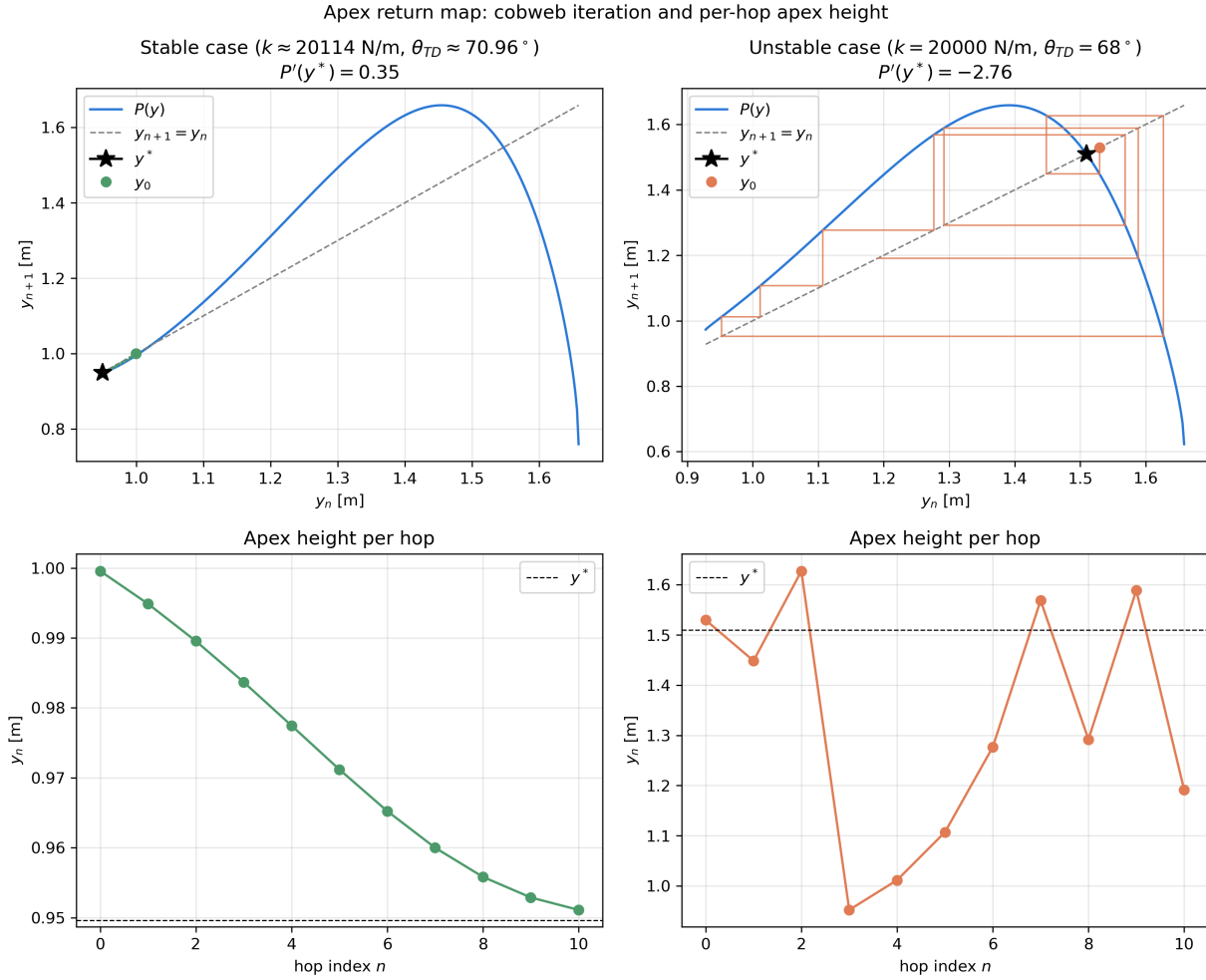


Figure 5: Apex-map cobweb iteration (top) and apex height per hop (bottom) for the stable cell of Section 4.4 (left, green) and the unstable point of Section 4.3 (right, orange). The stable case converges monotonically for a small perturbation; the unstable case diverges, alternating sides of  $y^*$  each hop.

## 4.6 Bifurcation Diagrams Along Each Sweep Direction

Section 4.4's grid already classifies every  $(k, \theta_{TD})$  cell as stable/unstable/no-fixed-point, but a single 2-D classification map does not show *how* the fixed point loses stability as a parameter varies continuously. `plotBifurcationDiagrams()` slices two 1-D bifurcation diagrams out of that same grid, both through the representative stable cell:  $y^*$  and  $P'(y^*)$  vs.  $\theta_{TD}$  at fixed  $k \approx 20114$  N/m, and vs.  $k$  at fixed  $\theta_{TD} \approx 70.96^\circ$  (Figure 6).

Both slices show the same structure:  $y^*(\theta_{TD})$  and  $y^*(k)$  vary smoothly and almost linearly, with no branching or discontinuity, so this is not a classical saddle-node or pitchfork bifurcation in  $y^*$ . Instead the loss of stability lives entirely in  $P'(y^*)$ : it crosses  $+1$  sharply at both edges of the narrow stable window (the thin green band already visible in Figure 3), confirming that the self-stable region is bounded by genuine  $|P'(y^*)| = 1$  crossings at both edges of the band, not by a grid-resolution artifact.

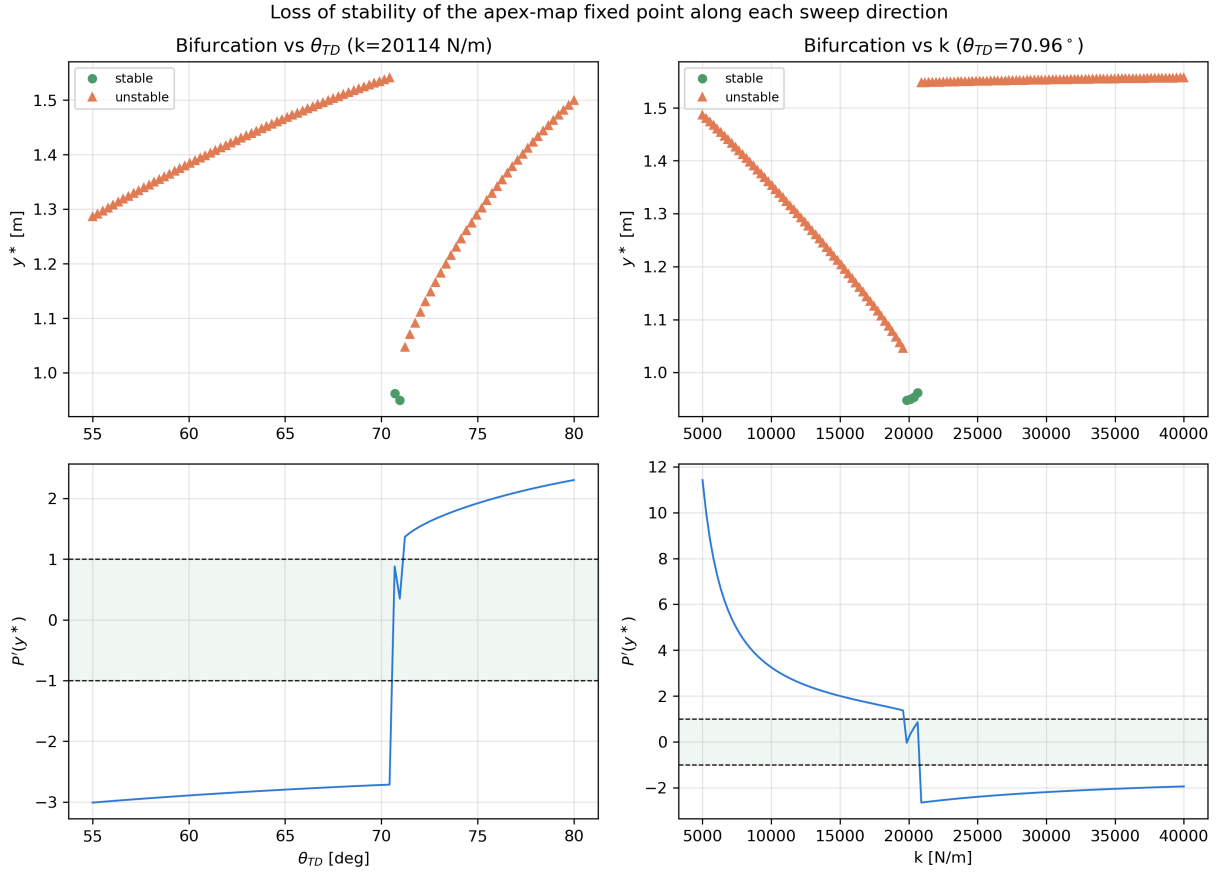


Figure 6: Bifurcation diagrams: fixed-point height  $y^*$  (top) and slope  $P'(y^*)$  (bottom) vs.  $\theta_{TD}$  at fixed  $k$  (left) and vs.  $k$  at fixed  $\theta_{TD}$  (right), sliced through the representative stable cell of Section 4.4. Shaded band:  $|P'(y^*)| < 1$  (stable). Marker shape doubles the stable/unstable color coding.

## 4.7 Stance-Phase Portrait

Section 1 motivates SLIP as building on the elastic pendulum studied in class. `stancePhasePortrait()` makes that connection concrete by plotting the stance-phase trajectory in the  $(r, \dot{r})$  and  $(\theta, \dot{\theta})$  phase planes for several touchdown angles at the same fixed apex energy (Figure 7). Each trajectory is an open arc, not a closed orbit: stance is a single finite-time pass through phase space between the touchdown event (circle) and the liftoff event (square), not a free-running periodic oscillator –  $r = r_0$  at both switching events, but  $\dot{r}$  changes sign in between as the leg compresses and rebounds. Steeper touchdown angles produce shorter, less compressed arcs that stay close to  $r = r_0$ , consistent with a more vertical landing imparting less compression, while shallower angles compress the leg further and sweep through a wider range of  $\theta$ .

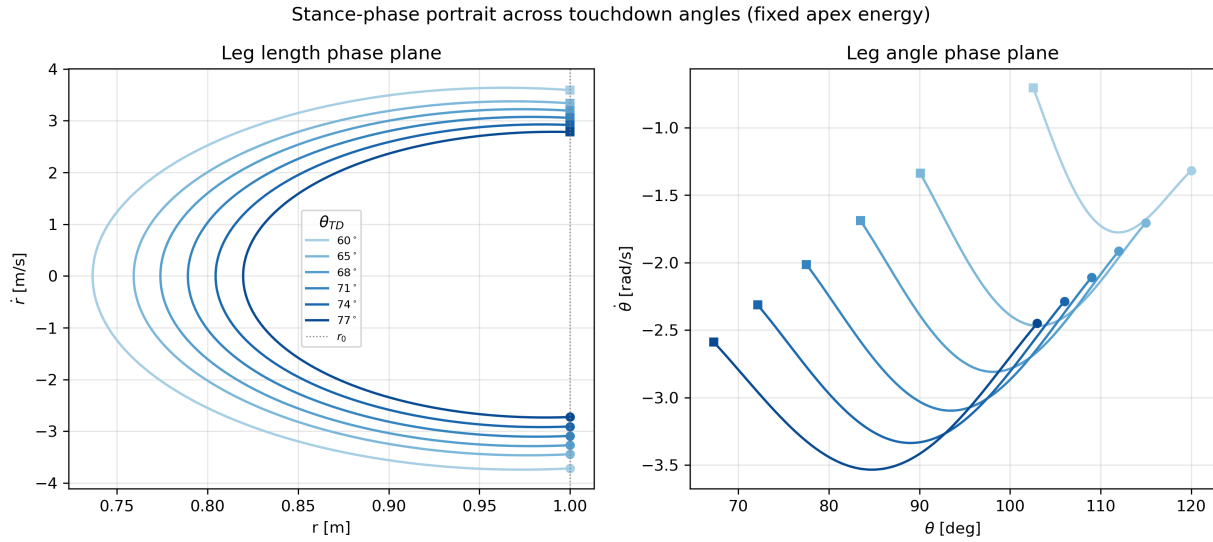


Figure 7: Stance-phase trajectories in the leg-length ( $r, \dot{r}$ ) and leg-angle ( $\theta, \dot{\theta}$ ) phase planes across touchdown angles  $\theta_{TD} \in [60^\circ, 77^\circ]$  at fixed apex energy. Circle = touchdown, square = liftoffStateSwitch.

## 5 Lessons Learned

This project was informative. It let me work through a multi-stage dynamic system and taught me how to handle state transition conditions clearly. It also expanded on our study of geometric dynamics analysis by requiring a different stability mechanism: instead of analyzing the whole system, we only need the points in space that best define the motion. It also drove home that finding a periodic solution is not the same as finding a stable one: the fixed point of the apex return map exists whether or not it is attracting, so stability has to be checked separately – and for the validated parameters, the hopping gait turned out not to be stable at all.

It was also a good project for working through robotics-style problems. The inverted pendulum is a famous model throughout robotics, but with the Lagrangian this derivation becomes relatively simple, opening the door to more accurate models. The parameter sweep also taught me a practical lesson about trusting numerical results: the first coarse grid over  $(k, \theta_{TD})$  turned up a stable band thin enough to be a grid-resolution artifact, and only after refining the grid and confirming the band held its width could I be confident the narrow self-stable region was a real feature of the model, not a numerical accident.

## References

- [1] R. Blickhan, “The spring-mass model for running and hopping,” *Journal of Biomechanics*, vol. 22, no. 11–12, pp. 1217–1227, 1989.
- [2] T. A. McMahon and G. C. Cheng, “The mechanics of running: how does stiffness couple with speed?,” *Journal of Biomechanics*, vol. 23, suppl. 1, pp. 65–78, 1990.
- [3] R. J. Full and D. E. Koditschek, “Templates and anchors: neuromechanical hypotheses of legged locomotion on land,” *Journal of Experimental Biology*, vol. 202, no. 23, pp. 3325–3332, 1999.
- [4] W. J. Schwind and D. E. Koditschek, “Approximating the stance map of a 2-DOF monoped runner,” *Journal of Nonlinear Science*, vol. 10, pp. 533–568, 2000.
- [5] U. Saranli, M. Buehler, and D. E. Koditschek, “RHex: a simple and highly mobile hexapod robot,” *International Journal of Robotics Research*, vol. 20, no. 7, pp. 616–631, 2001.
- [6] H. Geyer, A. Seyfarth, and R. Blickhan, “Compliant leg behaviour explains basic dynamics of walking and running,” *Proceedings of the Royal Society B*, vol. 273, no. 1603, pp. 2861–2867, 2006.

## Appendix: Code

`main.py` – touchdown map, stance-phase RHS, and the single-stance simulation and validation plots discussed in Section 4.1.

```

1 import copy
2 import time
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.integrate import solve_ivp
6 from scipy.optimize import least_squares
7 from scipy.optimize import brentq
8
9 # Paramerts
10 m = 80
11 r_o = 1
12 k = 20000
13 g = 9.81
14 thetaTD = np.radians(68)
15 y_o = 1.2
16 v_o = 3.0
17
18 params = {'m' : m , 'k' : k , 'g' : g, 'r_o' : r_o , 'y_o' : y_o}
19
20
21 # =====
22 # Stage 1 - Stance phase: touchdown -> standing -> liftoff
23 # First piece of the model: integrate a single stance phase and
24 # check that total mechanical energy is conserved through it.
25 # =====
26
27 def touchdownState(x_o_dot , y_o , theta_TD , r_o , g):
28     r = r_o
29     theta = np.arctan2(r_o * np.sin(theta_TD) , -r_o * np.cos(theta_TD))
30
31     x_TD_dot = x_o_dot
32     y_TD_dot = - np.sqrt(2*g*(y_o - r_o * np.sin(theta_TD)))
33
34     r_dot = x_TD_dot * np.cos(theta) + y_TD_dot * np.sin(theta)
35
36     theta_dot = (-x_TD_dot * np.sin(theta) + y_TD_dot * np.cos(theta)) / r_o

```

```

37     return [r , theta, r_dot , theta_dot]
38
39
40 def standingState(t , s, params):
41     r_dot = s[2]
42     theta_dot = s[3]
43
44     r_ddot = s[0] * theta_dot**2 - params['g'] * np.sin(s[1]) - (params['k'] /
45         params['m']) * (s[0] - params['r_o'])
46     theta_ddot = -(2/s[0]) * r_dot * theta_dot - (params['g'] / s[0]) * np.cos
47         (s[1])
48
49     return [r_dot , theta_dot, r_ddot, theta_ddot]
50
51 def liftoffStateSwitch(t, s, params):
52     return params['k'] * (s[0] - params['r_o'])
53
54 liftoffStateSwitch.terminal = True
55 liftoffStateSwitch.direction = 1
56
57 def energy(s, params):
58     E = (1/2) * params['m'] * (s[2]**2 + s[0]**2 * s[3]**2) + params['m'] *
59         params['g'] * s[0] * np.sin(s[1]) + (1/2) * params['k'] * (s[0] -
60         params['r_o'])**2
61
62     return E
63
64 # ----- run a single stance phase for diagnostics -----
65 # (mirrors the body of hop(), but keeps sol/s0 around for plotting)
66
67 def initialFunctionTest():
68     s0 = touchdownState(v_o, y_o, thetaTD, r_o, g)
69
70     t_span = (0.0, 1.0)
71     sol = solve_ivp(standingState, t_span, s0, args=(params,),
72         events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
73         dense_output=True, max_step=1e-3)
74
75     E = np.array([energy(sol.y[:, i], params) for i in range(sol.y.shape[1])])
76     drift = np.max(np.abs(E - E[0])) / np.abs(E[0])
77
78     # Plotting and data stuff, AI can fill in these gaps as simply learning to
79     # plot good is not hard and not my current goal
80
81     print("status:", sol.status)
82     print("liftoffStateSwitch time:", sol.t_events[0])
83     print("drift:", drift)
84
85     # ----- diagnostics -----
86     r_TD, th_TD, rdot_TD, thdot_TD = s0
87     s_LO = sol.y_events[0][0]
88     t_LO = sol.t_events[0][0]
89
90     print("--- touchdown state ---")
91     print(f" r      = {r_TD:.6f} m")
92     print(f" theta  = {np.degrees(th_TD):.4f} deg")
93     print(f" r_dot   = {rdot_TD:.6f} m/s")

```

```

88     print(f"   th_dot = {thdot_TD:.6f} rad/s")
89
90     print("--- liftoffStateSwitch state ---")
91     print(f"   status           = {sol.status}")
92     print(f"   stance duration= {t_L0:.6f} s")
93     print(f"   r                   = {s_L0[0]:.6f} m")
94     print(f"   theta              = {np.degrees(s_L0[1]):.4f} deg")
95     print(f"   r_dot              = {s_L0[2]:.6f} m/s")
96     print(f"   th_dot             = {s_L0[3]:.6f} rad/s")
97
98     print("--- validation ---")
99     print(f"   min leg length = {np.min(sol.y[0]):.6f} m")
100    print(f"   max compression= {params['r_o'] - np.min(sol.y[0]):.6f} m")
101    print(f"   E_0            = {E[0]:.6f} J")
102    print(f"   relative drift = {drift:.3e}")
103
104    # ----- figures -----
105    t = sol.t
106    r = sol.y[0]
107    th = sol.y[1]
108    x_com = r * np.cos(th)
109    y_com = r * np.sin(th)
110
111    fig, ax = plt.subplots(2, 2, figsize=(11, 8))
112
113    ax[0, 0].plot(t, r, color='C3')
114    ax[0, 0].axhline(params['r_o'], ls='--', lw=0.8, color='gray', label='
    $r_0$')
115    ax[0, 0].set_xlabel('t [s]')
116    ax[0, 0].set_ylabel('r [m]')
117    ax[0, 0].set_title('Leg length')
118    ax[0, 0].legend()
119
120    ax[0, 1].plot(t, np.degrees(th), color='C0')
121    ax[0, 1].axhline(90, ls='--', lw=0.8, color='gray', label='$90^\circ$')
122    ax[0, 1].set_xlabel('t [s]')
123    ax[0, 1].set_ylabel(r'$\theta$ [deg]')
124    ax[0, 1].set_title('Leg angle')
125    ax[0, 1].legend()
126
127    ax[1, 0].plot(x_com, y_com, color='C3', lw=2)
128    ax[1, 0].plot(0, 0, 'kv', ms=10, label='foot (pinned)')
129    ax[1, 0].plot(x_com[0], y_com[0], 'o', color='C0', label='touchdown')
130    ax[1, 0].plot(x_com[-1], y_com[-1], 's', color='C2', label='
    liftoffStateSwitch')
131    ax[1, 0].axhline(0, color='k', lw=1)
132    ax[1, 0].set_xlabel('x [m]')
133    ax[1, 0].set_ylabel('y [m]')
134    ax[1, 0].set_title('CoM trajectory (foot frame)')
135    ax[1, 0].axis('equal')
136    ax[1, 0].legend()
137
138    ax[1, 1].plot(t, E - E[0], color='C4')
139    ax[1, 1].set_xlabel('t [s]')
140    ax[1, 1].set_ylabel('$E - E_0$ [J]')
141    ax[1, 1].set_title(f'Energy drift (rel. {drift:.2e})')

```

```

142     ax[1, 1].ticklabel_format(axis='y', style='sci', scilimits=(0, 0))
143
144     for a in ax.flat:
145         a.grid(alpha=0.3)
146     fig.tight_layout()
147     fig.savefig(os.path.join(Script_DIR, 'stance_validation.png'), dpi=200)
148     plt.show()
149
150
151 # =====
152 # Stage 2 - Apex state: ballistic flight after liftoff
153 # Convert the liftoff (r, theta) state back to Cartesian (x, y)
154 # and propagate the free-flight trajectory forward to the apex
155 # (the point where the vertical velocity goes to zero).
156 # =====
157
158 def projectileApexState(s):
159     r_o = s[0]
160     theta_o = s[1]
161     r_o_dot = s[2]
162     theta_o_dot = s[3]
163
164     # convert state from liftoff back to cartesian
165     y_o = r_o * np.sin(theta_o)
166     x_o = r_o * np.cos(theta_o)
167
168     y_o_dot = r_o_dot * np.sin(theta_o) + theta_o_dot * r_o * np.cos(theta_o)
169     x_o_dot = r_o_dot * np.cos(theta_o) - theta_o_dot * r_o * np.sin(theta_o)
170
171     # Solve for when y velocity = 0, this is the highest point
172     t_apex = y_o_dot / g
173
174     # find corresponding x y x_dot y_dot values
175     y_dot = -g*t_apex + y_o_dot
176     x_dot = x_o_dot
177
178     y = y_o - ((g/2) * t_apex**2) + y_o_dot * t_apex
179     x = x_o + x_dot * t_apex
180
181     return [x, y, x_dot, y_dot]
182
183
184 # =====
185 # Stage 3 - Hop: touchdown -> stance -> liftoff -> apex
186 # Chains stages 1-2 together into a single map that takes an
187 # apex state and returns the next apex state, one hop later.
188 # =====
189
190 def hop(apexState, thetaTD, params):
191     s0 = touchdownState(apexState[2], apexState[1], thetaTD, params['r_o'],
192                         params['g'])
193     t_span = (0.0, 1.0)
194
195     sol = solve_ivp(standingState, t_span, s0, args=(params,),
196                     events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
197                     dense_output=True, max_step=1e-3)

```



```

197     s1 = projectileApexState(sol.y_events[0][0])
198
199     return s1
200
201 def energyHop(apexState, params):
202     E = (1/2) * params['m'] * (apexState[2]**2 + apexState[3]**2) + params['m',
203         ] * params['g'] * apexState[1]
204     return E
205
206
207 # =====
208 # Stage 4 - x is redundant: the apex-to-apex map is fully
209 # determined by total energy E and height y, not by x directly.
210 # =====
211
212 def proveX_dotIsRedundant():
213     # E = (1/2)*m*(x_dot^2 + y_dot^2) + m*g*y
214     # ((E - mgy) * 2/m - y_dot**2)**.5 = x_dot
215     # Y_dot = 0
216     # ((E-mgy)*2/m)**.5 = x_dot
217     # (2E/m - 2gy)**.5 = x_dot
218
219     initialState = [0,y_o,v_o,0]
220     initialThetaTD = thetaTD
221
222     E1 = energyHop(initialState , params)
223
224
225     finalState = hop(initialState, initialThetaTD, params)
226     x_dot_pred = np.sqrt(2*E1/ params['m'] - 2*params['g']*finalState[1])
227     E2 = energyHop(finalState, params)
228
229     print(f"Energy Drift over 1 hop: {E2-E1}")
230     print(f"X_dot Drift over 1 hop: {finalState[2]-x_dot_pred}")
231
232 # =====
233 # Stage 5 - Reduced 1-D apex map: fixed point and stability
234 # Stage 4 showed the hop only depends on E and y, so collapse the
235 # hop into a 1-D map y -> apexmap(y) and find the periodic apex
236 # height (fixed point), then check its local stability from the
237 # map's slope there.
238 # =====
239
240 def apexmap(y , thetaTD, params, E):
241     y = np.asarray(y).item()
242
243     x_dot = np.sqrt(2*E/ params['m'] - 2*params['g']*y)
244
245     inputApexState = [0, y , x_dot , 0]
246     outputApexState = hop(inputApexState, thetaTD, params)
247
248     return outputApexState[1]
249
250 # ----- find the fixed point of the apex map and its local stability
251 -----

```

```

251
252 def findApexMapFixedPoint(params, thetaTD, y_o=y_o, v_o=v_o):
253     initialState = [0, y_o, v_o, 0]
254     E = energyHop(initialState, params)
255     f = lambda y: apexmap(y, thetaTD, params, E) - y
256
257     # +.001 avoids singularities
258     y_min = params['r_o'] * np.sin(thetaTD) + .001 # apex must be at/above
259             touchdown height for this thetaTD
260     #  $2E/m - 2g \cdot y \geq 0$ 
261     #  $y < 2E / (2mg)$ 
262     y_max = 2 * E / (2 * params['m'] * params['g'])
263
264     y_sample = np.linspace(y_min, y_max, 30)
265     f_sample = np.array([f(yi) for yi in y_sample])
266
267     sign_changes = np.where(np.diff(np.sign(f_sample)) != 0)[0]
268
269     if sign_changes.size == 0:
270         raise ValueError(f"no sign change in apex map for thetaTD={thetaTD}, k
271             = {params['k']}")
272
273     y_min = y_sample[sign_changes[0]]
274     y_max = y_sample[sign_changes[0] + 1]
275
276     y_star: float = brentq(f, y_min, y_max) # type: ignore[assignment]
277
278     dy = max(1e-4 * abs(y_star), 1e-6)
279
280     apexmapPrime = (apexmap(y_star + dy, thetaTD, params, E) - apexmap(y_star
281         - dy, thetaTD, params, E)) / (2 * dy) # type: ignore
282
283     # print(apexmapPrime)
284
285     return (y_star, apexmapPrime)
286
287 # =====
288 # Stage 6 - Grid search: fixed point and stability over (k, thetaTD)
289 # Each grid point needs its own params dict (deep-copied off the
290 # base params) so sweeping 'k' never mutates the shared dict that
291 # other stages/functions read from.
292 # =====
293
294 def gridSearchFixedPoints(base_params, k_values, thetaTD_values, y_o=y_o, v_o=
295     v_o):
296     y_star_grid = np.full((len(k_values), len(thetaTD_values)), np.nan)
297     slope_grid = np.full((len(k_values), len(thetaTD_values)), np.nan)
298     n_failed = 0
299     n_total = len(k_values) * len(thetaTD_values)
300
301     # High-resolution sweeps can take well over an hour unattended, so print
302     # a per-row progress/ETA line rather than going silent until the end.
303     t_start = time.time()
304
305     for i, k_val in enumerate(k_values):

```

```

303     for j, theta_val in enumerate(thetaTD_values):
304         trial_params = copy.deepcopy(base_params)
305         trial_params['k'] = k_val
306         try:
307             y_star, slope = findApexMapFixedPoint(trial_params, theta_val,
308                                                     y_o, v_o)
309         except ValueError:
310             n_failed += 1
311             continue
312         y_star_grid[i, j] = y_star
313         slope_grid[i, j] = slope
314
315     n_done = (i + 1) * len(thetaTD_values)
316     elapsed = time.time() - t_start
317     eta = elapsed * (n_total - n_done) / n_done
318     print(f"    row {i+1}/{len(k_values)} (k={k_val:.0f}) done -- "
319           f"{n_done}/{n_total} cells, {elapsed/60:.1f} min elapsed, "
320           f"~{eta/60:.1f} min remaining")
321
322     print(f"{n_failed}/{n_total} cells failed to find a fixed point")
323     return y_star_grid, slope_grid
324
325
326 import numpy as np
327 import matplotlib.pyplot as plt
328 from matplotlib.colors import ListedColormap, BoundaryNorm
329
330 def plotStabilitySweep(k_values, thetaTD_values, y_star_grid, slope_grid,
331                       friction_floor_deg=59.0, save_path=None):
332     """
333     Classifies each grid cell as stable / unstable / no fixed point,
334     overlays the friction-cone floor, and saves the deliverable figure.
335     """
336     K, THETA = np.meshgrid(k_values, thetaTD_values, indexing='ij')
337
338     # 0 = no fixed point, 1 = unstable, 2 = stable
339     classification = np.zeros_like(slope_grid)
340     found = ~np.isnan(slope_grid)
341     classification[found & (np.abs(slope_grid) < 1)] = 2
342     classification[found & (np.abs(slope_grid) >= 1)] = 1
343
344     cmap = ListedColormap(['#d9d9d9', '#e07b54', '#4c9a6a']) # gray, orange,
345     green
346     bounds = [-0.5, 0.5, 1.5, 2.5]
347     norm = BoundaryNorm(bounds, cmap.N)
348
349     fig, ax = plt.subplots(figsize=(8, 6))
350     mesh = ax.pcolormesh(THETA, K, classification, cmap=cmap, norm=norm,
351                          shading='auto')
352
353     # friction-cone floor
354     ax.axvline(friction_floor_deg, color='k', linestyle='--', linewidth=1.5)
355     ax.axvspan(thetaTD_values.min(), friction_floor_deg, color='k', alpha
356                =0.08)
357     ax.text(friction_floor_deg + 0.5, k_values.max() * 0.97,

```

```

355         r'\theta_{TD} \geq %.0f^\circ$ (friction cone, $\mu=0.6$)' %
356             friction_floor_deg,
357         va='top', fontsize=9)
358
359 ax.set_xlabel(r'Touchdown angle $\theta_{TD}$ (deg)')
360 ax.set_ylabel(r'Leg stiffness $k$')
361 ax.set_title('Self-stable running region')
362
363 cbar = fig.colorbar(mesh, ax=ax, ticks=[0, 1, 2])
364 cbar.ax.set_yticklabels(['No fixed point', 'Unstable', 'Stable'])
365
366 fig.tight_layout()
367 if save_path:
368     fig.savefig(save_path, dpi=300)
369 return fig, ax
370
371 # =====
372 # Stage 7 - Local refinement:
373 # =====
374
375 def localRefinementSweep(base_params, k_center, theta_center_deg,
376                         k_half_width=2500.0, theta_half_width_deg=2.5,
377                         n=20, y_o=y_o, v_o=v_o):
378     """Rerun a small (k, thetaTD) window at finer resolution, zoomed on a
379     stable cell found by the coarse sweep."""
380     k_values = np.linspace(k_center - k_half_width, k_center + k_half_width, n)
381     thetaTD_deg = np.linspace(theta_center_deg - theta_half_width_deg,
382                              theta_center_deg + theta_half_width_deg, n)
383     thetaTD_values = np.radians(thetaTD_deg)
384
385     y_star_grid, slope_grid = gridSearchFixedPoints(base_params, k_values,
386                                                    thetaTD_values, y_o, v_o)
387     return k_values, thetaTD_deg, y_star_grid, slope_grid
388
389 def stableBandWidth(slope_grid):
390     """Longest run of contiguous stable ( $|\text{slope}| < 1$ ) cells found along any
391     single row (fixed k, varying thetaTD) or column (fixed thetaTD, varying
392     k). A width of 1 on a fine grid means the band really is that thin at
393     this resolution; >1 means the coarse sweep's one-cell staircase was an
394     artifact of the coarse spacing."""
395     stable = (~np.isnan(slope_grid)) & (np.abs(slope_grid) < 1)
396
397     def longest_run(mask_1d):
398         best = run = 0
399         for is_stable in mask_1d:
400             run = run + 1 if is_stable else 0
401             best = max(best, run)
402         return best
403
404     row_runs = [longest_run(row) for row in stable]
405     col_runs = [longest_run(col) for col in stable.T]
406     return max(row_runs, default=0), max(col_runs, default=0)
407

```

```

408
409 # =====
410 # Stage 8 - Deliverable figures: apex-map iteration (cobweb),
411 # bifurcation diagrams sliced from the (k, thetaTD) sweep, and
412 # the stance-phase portrait. These answer the write-up's own
413 # open questions about the sweep (Section 4.4's insert box):
414 # what does convergence/divergence of the apex map actually look
415 # like, and where exactly does the fixed point lose stability?
416 # =====
417
418 def apexMapDomain(thetaTD, params, E):
419     """Same y-domain bounds used inside findApexMapFixedPoint(): apex height
420     must be at/above the touchdown height for this thetaTD, and low enough
421     that  $2E/m - 2*g*y$  stays non-negative."""
422     y_min = params['r_o'] * np.sin(thetaTD) + 1e-3
423     y_max = E / (params['m'] * params['g'])
424     return y_min, y_max
425
426
427 def iterateApexMap(y0, thetaTD, params, E, n_iter=10, domain=None):
428     """Iterates  $y_{(n+1)} = \text{apexmap}(y_n)$  starting from y0, stopping early if an
429     iterate leaves the physically valid domain -- a real possibility right
430     next to an unstable fixed point, where the map blows up in a few hops."""
431     y_min, y_max = domain if domain is not None else (-np.inf, np.inf)
432     ys = [y0]
433     for _ in range(n_iter):
434         y_prev = ys[-1]
435         if not (y_min < y_prev < y_max):
436             break
437         try:
438             y_next = apexmap(y_prev, thetaTD, params, E)
439         except Exception:
440             break
441         if not np.isfinite(y_next):
442             break
443         ys.append(y_next)
444     return np.array(ys)
445
446
447 def _apexMapIterationPanel(ax_cobweb, ax_trace, thetaTD, params, E, y_star,
448     slope,
449                             y0_offset, label, color, n_iter=10):
450     """One stable- or unstable-case pair: a cobweb diagram of  $P(y)$  with the
451     staircase iteration overlaid, plus the same iteration as  $y_n$  vs hop
452     index. Each panel is self-labeled (title + legend), so color here is
453     never the only channel identifying stable vs. unstable."""
454     y_min, y_max = apexMapDomain(thetaTD, params, E)
455     y_curve = np.linspace(y_min, y_max, 300)
456     P_curve = np.array([apexmap(y, thetaTD, params, E) for y in y_curve])
457     ax_cobweb.plot(y_curve, P_curve, color='#2a78d6', lw=1.5, label='$P(y)$')
458     ax_cobweb.plot(y_curve, y_curve, color='gray', ls='--', lw=1, label='$y_{n+1}=y_n$')
459     ax_cobweb.plot(y_star, y_star, marker='*', color='k', ms=12, zorder=5,
460         label='$y^*$')
460

```

```

461     ys = iterateApexMap(y_star + y0_offset, thetaTD, params, E, n_iter=n_iter,
462                        domain=(y_min, y_max))
463     for n in range(len(ys) - 1):
464         ax_cobweb.plot([ys[n], ys[n]], [ys[n], ys[n + 1]], color=color, lw=1,
465                        alpha=0.85)
466         ax_cobweb.plot([ys[n], ys[n + 1]], [ys[n + 1], ys[n + 1]], color=color
467                        , lw=1, alpha=0.85)
468     ax_cobweb.plot(ys[0], ys[0], 'o', color=color, ms=6, label='$y_0$')
469     ax_cobweb.set_xlabel('$y_n$ [m]')
470     ax_cobweb.set_ylabel('$y_{n+1}$ [m]')
471     ax_cobweb.set_title(f"{label}\n${P}(y^*) = {slope:.2f}$")
472     # fontsize=7 in source units reads fine at native 3300x2700 resolution,
473     # but this figure is placed at 0.95\linewidth on a letter page -- a
474     # ~0.59x downscale that shrinks it to under 5pt on the printed/rendered
475     # page, below any legible threshold. Bumped to match the other panel
476     # text (default ~10pt) so it survives that downscale.
477     ax_cobweb.legend(fontsize=10, loc='best')
478     ax_cobweb.grid(alpha=0.3)
479
480     ax_trace.plot(range(len(ys)), ys, 'o-', color=color)
481     ax_trace.axhline(y_star, color='k', ls='--', lw=0.8, label='$y^*$')
482     ax_trace.set_xlabel('hop index $n$')
483     ax_trace.set_ylabel('$y_n$ [m]')
484     ax_trace.set_title('Apex height per hop')
485     ax_trace.legend(fontsize=10)
486     ax_trace.grid(alpha=0.3)
487     return ys
488
489 def plotApexMapIteration(stable_case, unstable_case, save_path=None):
490     """Side-by-side cobweb + iteration-trace figure contrasting the stable
491     fixed point (Section 4.4/4.5's representative cell) with the unstable
492     one (Section 4.3's validated parameter set) -- directly answers the
493     write-up's request for an iteration plot showing convergence."""
494     fig, axes = plt.subplots(2, 2, figsize=(11, 9))
495
496     _apexMapIterationPanel(axes[0, 0], axes[1, 0], color='#4c9a6a', **
497                           stable_case)
498     _apexMapIterationPanel(axes[0, 1], axes[1, 1], color='#e07b54', **
499                           unstable_case)
500
501     fig.suptitle('Apex return map: cobweb iteration and per-hop apex height')
502     fig.tight_layout()
503     if save_path:
504         fig.savefig(save_path, dpi=300)
505     return fig, axes
506
507 def _bifurcationColumn(ax_top, ax_bot, x, y_star, slope, xlabel, title):
508     found = ~np.isnan(slope)
509     stable = found & (np.abs(slope) < 1)
510     unstable = found & (np.abs(slope) >= 1)
511
512     # Marker shape (o vs ^), not just color, distinguishes stable/unstable --
513     # this green/orange pair reads fine for normal vision but is a poor
514     # protanopia/deutanopia pairing, so identity can't ride on hue alone.

```

```

512     ax_top.plot(x[stable], y_star[stable], 'o', color='#4c9a6a', ms=5, label='
        stable')
513     ax_top.plot(x[unstable], y_star[unstable], '^', color='#e07b54', ms=5,
        label='unstable')
514     ax_top.set_ylabel(r'$y^{\ast}$ [m]')
515     ax_top.set_title(title)
516     ax_top.legend(fontsize=8)
517     ax_top.grid(alpha=0.3)
518
519     ax_bot.plot(x[found], slope[found], color='#2a78d6', lw=1.2)
520     ax_bot.axhline(1, ls='--', color='k', lw=0.8)
521     ax_bot.axhline(-1, ls='--', color='k', lw=0.8)
522     ax_bot.axhspan(-1, 1, color='#4c9a6a', alpha=0.08)
523     ax_bot.set_xlabel(xlabel)
524     ax_bot.set_ylabel(r"$P'(y^{\ast})$")
525     ax_bot.grid(alpha=0.3)
526
527
528 def plotBifurcationDiagrams(k_values, thetaTD_deg_values, y_star_grid,
    slope_grid,
529                             i_fixed_k, j_fixed_theta, save_path=None):
530     """Two one-parameter bifurcation slices through the (k, thetaTD) sweep
531     grid, both through the same representative stable cell used for the
532     local refinement in Stage 7: y* (top) and P'(y*) (bottom) vs. thetaTD at
533     fixed k (left column), and vs. k at fixed thetaTD (right column). Where
534     the bottom row crosses +/-1, the fixed point above loses stability -- a
535     bifurcation of the apex map."""
536     k_fixed_val = k_values[i_fixed_k]
537     theta_fixed_val = thetaTD_deg_values[j_fixed_theta]
538
539     fig, axes = plt.subplots(2, 2, figsize=(11, 8))
540
541     _bifurcationColumn(axes[0, 0], axes[1, 0], thetaTD_deg_values,
542                        y_star_grid[i_fixed_k, :], slope_grid[i_fixed_k, :],
543                        xlabel=r'$\theta_{TD}$ [deg]',
544                        title=f'Bifurcation vs $\theta_{TD}$ (k={
545                             k_fixed_val:.0f} N/m)')
546     _bifurcationColumn(axes[0, 1], axes[1, 1], k_values,
547                        y_star_grid[:, j_fixed_theta], slope_grid[:,
548                        j_fixed_theta],
549                        xlabel='k [N/m]',
550                        title=f'Bifurcation vs k ($\theta_{TD}$={
551                             theta_fixed_val:.2f}$^\circ$)')
552
553     fig.suptitle('Loss of stability of the apex-map fixed point along each
554     sweep direction')
555     fig.tight_layout()
556     if save_path:
557         fig.savefig(save_path, dpi=300)
558     return fig, axes
559
560 def stancePhasePortrait(thetaTD_deg_list, y_o, v_o, params, save_path=None):
561     """Runs one stance phase per touchdown angle at fixed apex energy (same
562     y_o, v_o for all) and plots the resulting trajectories in the (r, r_dot)
563     and (theta, theta_dot) phase planes -- the SLIP-stance analogue of the

```

```

561 elastic-pendulum phase portraits from lecture. Circle = touchdown,
562 square = liftoffStateSwitch. thetaTD is a single-hue sequential
563 parameter here (not a category), so it gets a light->dark blue ramp
564 rather than a categorical palette."""
565 fig, axes = plt.subplots(1, 2, figsize=(11, 5))
566 cmap = plt.get_cmap('Blues')
567 n = len(thetaTD_deg_list)
568
569 for idx, theta_deg in enumerate(thetaTD_deg_list):
570     theta_rad = np.radians(theta_deg)
571     color = cmap(0.35 + 0.55 * idx / max(n - 1, 1))
572     s0 = touchdownState(v_o, y_o, theta_rad, params['r_o'], params['g'])
573     sol = solve_ivp(standingState, (0.0, 1.0), s0, args=(params,),
574                     events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
575                     dense_output=True, max_step=1e-3)
576     if sol.status != 1 or len(sol.t_events[0]) == 0:
577         continue
578     r, th, rdot, thdot = sol.y
579     axes[0].plot(r, rdot, color=color, lw=1.5, label=f'{theta_deg:.0f}$^\backslash$
580         circ$')
581     axes[0].plot(r[0], rdot[0], 'o', color=color, ms=5)
582     axes[0].plot(r[-1], rdot[-1], 's', color=color, ms=5)
583
584     axes[1].plot(np.degrees(th), thdot, color=color, lw=1.5)
585     axes[1].plot(np.degrees(th[0]), thdot[0], 'o', color=color, ms=5)
586     axes[1].plot(np.degrees(th[-1]), thdot[-1], 's', color=color, ms=5)
587
588     axes[0].axvline(params['r_o'], ls=':', color='gray', lw=0.8, label='$r_0$')
589
590     axes[0].set_xlabel('r [m]')
591     axes[0].set_ylabel(r'$\dot{r}$ [m/s]')
592     axes[0].set_title('Leg length phase plane')
593     axes[0].legend(title=r'$\theta_{TD}$', fontsize=7)
594     axes[0].grid(alpha=0.3)
595
596     axes[1].set_xlabel(r'$\theta$ [deg]')
597     axes[1].set_ylabel(r'$\dot{\theta}$ [rad/s]')
598     axes[1].set_title('Leg angle phase plane')
599     axes[1].grid(alpha=0.3)
600
601     fig.suptitle('Stance-phase portrait across touchdown angles (fixed apex
602         energy)')
603     fig.tight_layout()
604     if save_path:
605         fig.savefig(save_path, dpi=300)
606     return fig, axes
607
608 # =====
609 # Run
610 # =====
611
612 import os
613 import warnings
614
615 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

```



```

614
615 # All Stage 8 deliverable figures are saved directly into the latex source
616 # tree so there is no manual copy step before compiling.
617 GRAPH_DIR = os.path.normpath(os.path.join(SCRIPT_DIR, '..', 'latex', 'Figures'
618 , 'Graphs'))
619 os.makedirs(GRAPH_DIR, exist_ok=True)
620
621 RESULTS_PATH = os.path.join(SCRIPT_DIR, "sweep_results.npz")
622 RERUN_SWEEP = False # set True to force a fresh sweep even if a cached file
623 exists
624
625 def runOrLoadSweep(base_params, k_values, thetaTD_values, y_o, v_o,
626 results_path=RESULTS_PATH, rerun=RERUN_SWEEP):
627     if not rerun and os.path.exists(results_path):
628         data = np.load(results_path)
629         cached_k = data['k_values']
630         cached_theta = data['thetaTD_values']
631         if (cached_k.shape == k_values.shape and np.allclose(cached_k,
632 k_values)
633             and cached_theta.shape == thetaTD_values.shape
634             and np.allclose(cached_theta, thetaTD_values)):
635             print(f"Loaded cached sweep from {results_path}")
636             return data['y_star_grid'], data['slope_grid']
637         else:
638             print("Cached grid does not match requested k_values/
639 thetaTD_values - rerunning.")
640
641     print("Running fresh sweep...")
642     y_star_grid, slope_grid = gridSearchFixedPoints(base_params, k_values,
643 thetaTD_values, y_o, v_o)
644     np.savez(results_path,
645 k_values=k_values,
646 thetaTD_values=thetaTD_values,
647 y_star_grid=y_star_grid,
648 slope_grid=slope_grid)
649     print(f"Saved sweep to {results_path}")
650     return y_star_grid, slope_grid
651
652 if __name__ == "__main__":
653     warnings.filterwarnings('error', category=np.exceptions.ComplexWarning)
654
655     # Bumped from 40x40: the local refinement check below showed the coarse
656     # grid's one-cell-wide stable "staircase" was a resolution artifact, not
657     # a real measure-zero band -- it thickened into a connected patch at 4x
658     # finer spacing. This resolution matches that refinement window's cell
659     # spacing (~0.26 deg, ~265 stiffness) across the full sweep range.
660     # ~12,600 cells; at ~0.47 s/cell (measured on this machine) that's
661     # roughly 90-100 minutes. Grab coffee.
662     k_sweep = np.linspace(5000, 40000, 133)
663     thetaTD_sweep_deg = np.linspace(55, 80, 95)
664     thetaTD_sweep_rad = np.radians(thetaTD_sweep_deg)
665
666     y_star_grid, slope_grid = runOrLoadSweep(params, k_sweep,
667 thetaTD_sweep_rad, y_o, v_o)
668     print("y_star grid:\n", y_star_grid)

```

```

664 print("slope grid:\n", slope_grid)
665
666 fig, ax = plotStabilitySweep(k_sweep, thetaTD_sweep_deg, y_star_grid,
667                             slope_grid,
668                             friction_floor_deg=59.0,
669                             save_path=os.path.join(SCRIPT_DIR, '
670                                                         stability_sweep.png'))
671
672 coarse_row_run, coarse_col_run = stableBandWidth(slope_grid)
673 print(f"coarse grid stable band width: {coarse_row_run} cell(s) along
674       theta, "
675       f"{coarse_col_run} cell(s) along k")
676
677 # ---- local refinement: zoom a 20x20 window (5 deg x 5000 stiffness,
678 # matching one dip below the friction-cone floor at 59 deg so we're not
679 # zooming into a physically-excluded corner) onto a stable cell from the
680 # coarse sweep and see if the band thickens at 4x the resolution ----
681 stable_coarse = (~np.isnan(slope_grid)) & (np.abs(slope_grid) < 1)
682 stable_idx = np.argwhere(stable_coarse)
683 if stable_idx.size == 0:
684     print("no stable cells found in the coarse sweep -- skipping
685           refinement")
686 else:
687     i, j = stable_idx[len(stable_idx) // 2] # a representative stable
688     cell, not an edge case
689     k_center = k_sweep[i]
690     theta_center_deg = thetaTD_sweep_deg[j]
691     print(f"refining around k={k_center:.1f}, thetaTD={theta_center_deg:.2
692           f} deg")
693
694     k_fine, theta_fine_deg, y_star_fine, slope_fine = localRefinementSweep
695     (
696         params, k_center, theta_center_deg,
697         k_half_width=2500.0, theta_half_width_deg=2.5, n=20, y_o=y_o, v_o=
698         v_o)
699
700     fine_row_run, fine_col_run = stableBandWidth(slope_fine)
701     print(f"refined grid stable band width: {fine_row_run} cell(s) along
702           theta, "
703           f"{fine_col_run} cell(s) along k (out of {len(theta_fine_deg)}x{
704               len(k_fine)} cells, "
705           f"~{(theta_fine_deg[1]-theta_fine_deg[0]):.3f} deg / {(k_fine
706               [1]-k_fine[0]):.1f} stiffness spacing)")
707
708     if fine_row_run <= 1 and fine_col_run <= 1:
709         print("band stays one-cell-wide at 4x finer resolution -- looks
710               like a real, "
711               "genuinely narrow stability margin, not a resolution
712               artifact.")
713     else:
714         print("band thickens at finer resolution -- the coarse sweep's
715               staircase was "
716               "a grid artifact; bump the full sweep's resolution before
717               finalizing.")
718
719     plotStabilitySweep(k_fine, theta_fine_deg, y_star_fine, slope_fine,

```

```

705         friction_floor_deg=59.0,
706         save_path=os.path.join(SCRIPT_DIR, '
              stability_refinement.png'))
707
708     # ---- Stage 8: deliverable figures for the write-up ----
709     trial_stable_params = copy.deepcopy(params)
710     trial_stable_params['k'] = k_center
711     y_star_stable = y_star_grid[i, j]
712     slope_stable = slope_grid[i, j]
713
714     y_star_unstable, slope_unstable = findApexMapFixedPoint(params,
              thetaTD)
715     E_fixed = energyHop([0, y_o, v_o, 0], params)
716
717     stable_case = dict(
718         thetaTD=np.radians(theta_center_deg), params=trial_stable_params,
              E=E_fixed,
719         y_star=y_star_stable, slope=slope_stable, y0_offset=0.05,
720         label=f'Stable case ($k\\approx{k_center:.0f}$ N/m, '
              f'$\\theta_{{TD}}\\approx{theta_center_deg:.2f}^\\circ$'))
721     unstable_case = dict(
722         thetaTD=thetaTD, params=params, E=E_fixed,
723         y_star=y_star_unstable, slope=slope_unstable, y0_offset=0.02,
724         label=f'Unstable case ($k={params["k"]:.0f}$ N/m, '
              f'$\\theta_{{TD}}={np.degrees(thetaTD):.0f}^\\circ$'))
725
726     plotApexMapIteration(stable_case, unstable_case,
727         save_path=os.path.join(GRAPH_DIR, '
              apex_map_iteration.png'))
728
729     plotBifurcationDiagrams(k_sweep, thetaTD_sweep_deg, y_star_grid,
              slope_grid, i, j,
730         save_path=os.path.join(GRAPH_DIR, '
              bifurcation_diagrams.png'))
731
732     stancePhasePortrait([60, 65, 68, 71, 74, 77], y_o, v_o, params,
733         save_path=os.path.join(GRAPH_DIR, '
              stance_phase_portrait.png'))
734
735     print(f"Stage 8 figures saved to {GRAPH_DIR}")
736
737

```